# AdsKG-Agent: Project Documentation

AI Engineering Cohort capstone · Anshika Chaurasia (solo) · 30 October 2026 · code, benchmark and all run logs in the repository (README has every number and command)

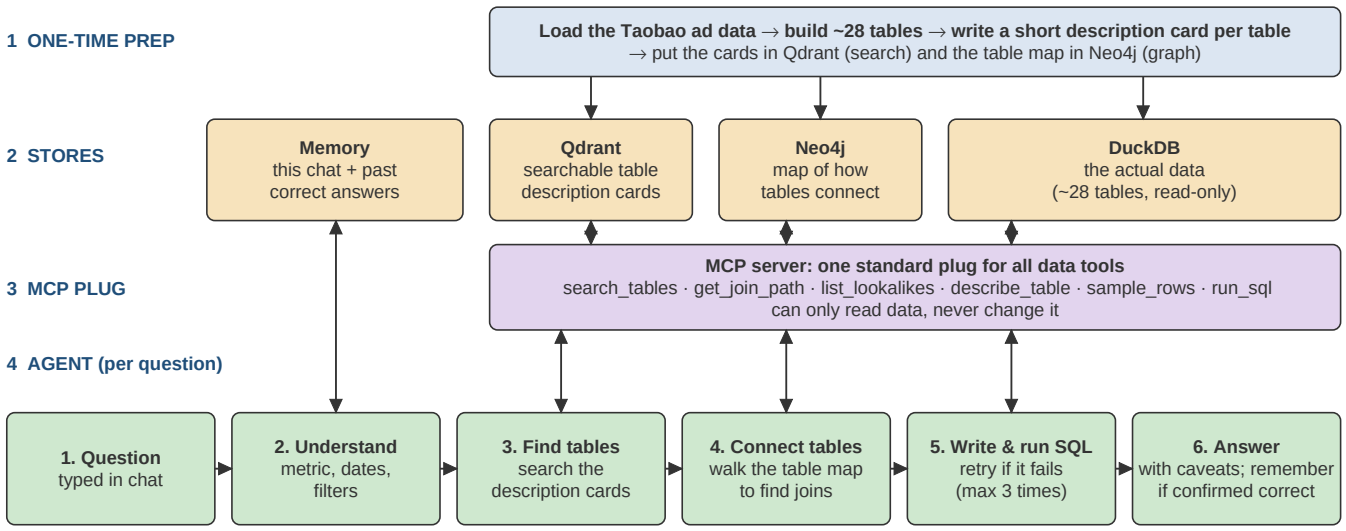
## 1. Problem

Ad teams ask "what happened" questions ("CTR of campaign 143747 yesterday, excluding suspicious clicks?"). The hard part is not writing SQL. It's **picking the right table among look-alikes** (hourly vs daily, all vs valid traffic, impressions vs unique people, live vs final) and **finding the lookup tables to join**. **Input:** one English question (or a follow-up). **Output:** a correct answer, the SQL that produced it and the tables used, or "not sure". **Scope:** the "what happened" stage only, read-only, one warehouse.

## 2. Data Processing Pipeline

| Step | What happens | Output / check |
|---|---|---|
| Ingest | Taobao Ad Display/Click (Kaggle): 26M impressions, ad features, user profiles. Loaded into DuckDB; Unix time → China time; brand "NULL" fixed; written rules (*now* = 13 May 23:59, suspicious = >5 clicks by one user in one hour) | `warehouse/build.py`, rules.md |
| Model | 28 tables in look-alike families: dimensions, one event fact, hourly/daily/weekly aggregates, valid-traffic versions, audience splits, reach (unique users), live last-6-hours tables | 28 tables, ~15 s build |
| Describe | One card per table: description, grain, freshness, traffic, columns, look-alikes and how they differ | `catalog/cards/*.yaml` |
| Index | Cards → Qdrant (dense bge-small + BM25 sparse). Table map → Neo4j: tables, columns, metrics, business words, JOINS_WITH / LOOKALIKE_OF / ROLLS_UP_TO | search 10/10 in top 5; map 10/10 routes |
| Distractors | For the bigger-catalog test: 222 extra tables (TPC-DS, TPC-H, BIRD), cards drafted by AI in the same style, added to Qdrant and Neo4j only | catalogs of 28 / 100 / 250 tables |
| Benchmark | 120 questions (40 easy / 40 medium / 40 tricky; 40 practice / 80 test) + 20 three-turn conversations, from 30 templates; gold SQL checked against raw data (28/28 agree); frozen before B–E | fingerprint 61e282210933c887 |

## 3. Architecture: five versions, one idea added at a time




*Agent rules (the "harness"): at most 8 steps · at most 3 retries · cost limit per question · says "not sure" instead of guessing*

*Figure 1. Prep once; per question the agent finds and connects tables through the MCP server, runs SQL and answers. Built as designed, except Langfuse (logs are written to JSON files instead).*

| Version | Retrieval / orchestration | What it tests |
|---|---|---|
| A. No search | all 28 cards in the prompt, one LLM call | baseline: is retrieval needed at all? |
| B. Basic search | dense search, top 5 cards | vanilla RAG |
| C. Smart search | B + query rewrite + hybrid (dense + BM25, RRF) + cross-encoder re-rank; each switchable | advanced RAG, with ablation |
| D. Search + map | B's 5 hits + Neo4j shortest join paths and look-alikes, at most 8 cards | GraphRAG |
| E. Agent | LangGraph ReAct loop over 6 MCP tools; chat memory; optional example memory | agent + memory |

## 4. Retrieval Design

- **Cards, not raw schemas:** each card states grain, freshness and traffic filter, so look-alikes can be told apart. Search alone still ranks a look-alike first in 5 of 10 checks: these tables are near-duplicates by design.
- **Graph expansion (D):** from the search hits, add the shortest JOINS_WITH paths and LOOKALIKE_OF neighbours, capped at 8 cards. It raised "needed tables shown" from 62% to 79% on test.
- **Advanced RAG (C) did not help:** on test, all three tricks 68.3%; without rewrite 77.5%; without re-rank 76.3%; without hybrid 72.5%. The rewrite pushes the query toward one summary table, and the re-ranker scores tables one at a time, so neither brings in the lookup table that a join needs.

## 5. Orchestration and Context Handling (version E)

| Part | Design |
|---|---|
| Loop | LangGraph StateGraph: model → tools → model ... until an answer. Limits: 8 steps, 3 failed SQL runs, 60k input tokens. At a limit it must answer only from SQL that already ran, or say "not sure" |
| Tools (MCP) | One FastMCP server, `adskg`: search_tables, get_join_path, list_lookalikes, describe_table, sample_rows, run_sql. The agent discovers them over stdio, like any MCP client |
| Safety | Inside the server: read-only DB with file access switched off and locked, exactly one SELECT (sqlglot), only the 28 warehouse tables, 1,000 rows, 20 s timeout, every call logged. 12/12 attacks blocked, incl. file reads like read_text('.env'), a gap found and fixed after the final runs (no result changed) |

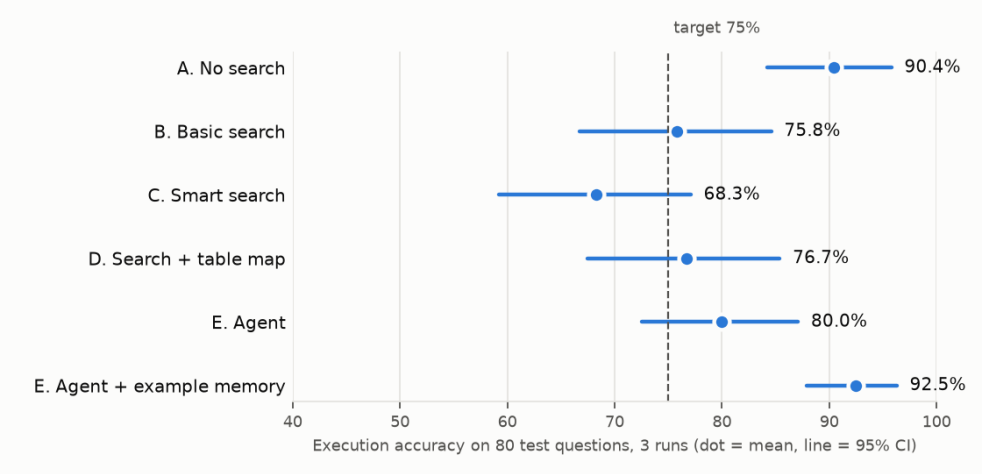
| Part | Design |
|---|---|
| Context | Tool results kept under 4,000 characters (long results drop list items instead of being cut); only the last 6 user turns sent; prompt states data facts (IDs only, how events join to lookups) and working habits (search at most twice, query early) |
| Memory | **Chat memory:** checkpointer per conversation thread. **Example memory:** 34 correctly answered *practice* questions; the 2 most similar (cosine ≥ 0.75) are added to the prompt. Test questions can never be saved (the harness refuses) |

## 6. Evaluation Method

- **Execution accuracy** (main score): result rows must contain the gold rows (order ignored, 4-decimal rounding, extra columns allowed). **Table F1** vs gold tables; **needed tables shown** (search recall).
- **Reliability:** 80 test questions × 3 runs per version; 95% bootstrap CIs over questions; exact McNemar test (right in ≥ 2 of 3 runs). Practice questions were used for all building and fixing.
- **LLM-as-a-judge:** gpt-5 grades every answer of the first run of each version, blind to version and gold: relevance, caveats, faithfulness (1–5), acceptable yes/no. **Human check:** I graded 30 random answers (version hidden, reference query visible): same verdict as the AI grader on 29/30, **Cohen's kappa = 0.65** (target 0.6). My first pass accepted all 30 (kappa 0.00); two grades (items 19 and 24) were changed to "no" after re-checking them against the reference. Disclosed in the README.
- **Failure analysis:** all 91 wrong answers labelled by type (gpt-5, 24 of 27 labels confirmed by hand).

## 7. Results (80 test questions, 3 runs, gpt-5-mini)

| Version | Accuracy [95% CI] | Medium | Tricky | Table F1 | Tables shown | Median s | Median $ | Judge OK |
|---|---|---|---|---|---|---|---|---|
| A. No search | 90.4% [84–96] | 87.7% | 83.3% | 0.96 | 100.0% | 3.2 | 0.0012 | 96.3% |
| B. Basic search | 75.8% [67–85] | 48.1% | 79.5% | 0.67 | 62.5% | 3.3 | 0.0006 | 76.2% |
| C. Smart search | 68.3% [59–77] | 38.3% | 75.6% | 0.67 | 57.5% | 9.4 | 0.0011 | 73.8% |
| D. Search + map | 76.7% [68–85] | 51.9% | 80.8% | 0.79 | 78.7% | 3.3 | 0.0006 | 78.7% |
| E. Agent | 80.0% [73–87] | 54.3% | 92.3% | 0.81 | 82.1% | 8.2 | 0.0019 | 92.5% |
| **E + example memory** | 92.5% [88–96] | 82.7% | 98.7% | 0.85 | 80.8% | 8.6 | 0.0019 | 92.5% |




- **Significant (p < 0.05):** A > B (p = 0.0063), A > D (p = 0.001), E + memory > E (15 fixed, 0 broken, p = 0.0001). With 8 comparisons, the Bonferroni check (p < 0.00625) keeps A > D and memory > no memory; A > B is borderline. **Not significant:** B vs C vs D, D vs E, A vs E, and **A vs E + memory (p = 0.11): statistically tied**.
- **Follow-ups** (15 conversations × 3 turns): chat memory on 89.6%, off 28.1%; turns 2–3 drop to 0% without memory.
- **Bigger catalog** (1 run per size): A stays at 90% at 250 tables but its prompt grows from 3k to 29k tokens (cost ×6); B/C/D cost the same at every size; D 76.7% → 73.8% (target: drop ≤ 10 points). The extra tables are mostly off-topic, so this shows cost growing more than confusion.
- **Memory caveat:** 56 of 80 test questions share a template with a practice example in memory. On the other 24, memory still lifts E from 72% to 85%. No test question is in memory.
- **Post-hoc, after the final runs:** two agent bugs (long results cut mid-JSON counted as failed runs; SQL shown but never run scored as "not sure") were fixed and the agent re-run: E 80.0% → 84.2%, E + memory 92.5% → 95.0% (still tied with A, p = 0.11). Reported apart from the frozen results, because the bugs were found by reading test answers.
- **All 7 design-doc targets met** (kappa 0.65, see section 6): best accuracy ≥ 75%; +27 points over B on hard questions (target +10); table F1 0.85; follow-ups 90% (target 80%); 8.6 s and $0.002 per question; D drop 2.9 points.

## 8. Failure Analysis, Attempted Approaches and Pivots

| Failure type (first test run) | A | B | C | D | E | E+mem | Example |
|---|---|---|---|---|---|---|---|
| Wrong calculation | 5 | 6 | 4 | 7 | 2 | 2 | audience table has a row per age level; not summed |
| Missing connecting table | 0 | 5 | 10 | 3 | 0 | 0 | "high-spending users" without joining dim_user |
| Strict grading (right value) | 3 | 3 | 2 | 3 | 6 | 0 | weekly answer without days_in_week; ranked list |
| Wrong look-alike | 0 | 4 | 5 | 2 | 2 | 0 | valid-traffic table when all traffic was asked |
| Not sure (no SQL) | 0 | 0 | 1 | 2 | 6 | 4 | agent ran out of steps |
| Other (time window, join, column) | 0 | 1 | 1 | 0 | 1 | 1 | "today so far" from the 6-hour live table |
| **Wrong answers** | 8 | 19 | 23 | 17 | 17 | 7 | |

| Tried | What went wrong | Pivot |
|---|---|---|
| D from 3 search hits | lost tables B had found (practice 77.5%) | start from B's 5 hits (85%) |
| langchain-mcp-adapters | needs mcp < 2; breaks FastMCP 4 | 25-line bridge on FastMCP's client |
| E answering at the step limit | wrote SQL it never ran, with invented columns | grade only SQL that ran; else "not sure" |
| E with a short prompt | spent its steps hunting for names and ready-made tables | data facts + habits in prompt (75% → 85%) |

| Tried | What went wrong | Pivot |
|---|---|---|
| Advanced RAG (C) | worse than basic search on practice and test | reported as a negative result; not tuned |
| Sequential final runs | 3.5 h; local Qdrant allows one process | one index copy per parallel chain (2 h) |

## 9. Final Choice and Framework Justification

**At 28 tables, A and E + example memory are statistically tied** (90.4% vs 92.5%, p = 0.11); A is simpler and 2.7× faster. **For one-shot questions on a small catalog, use A. The project ships E + example memory** because the real use needs follow-ups (90% vs 0% for turns 2–3 without chat memory) and a growing catalog (it retrieves through tools instead of reading every card); it is significantly better than the agent without memory, and nearly always faithful to the data (judge 4.8/5; one invented "SQL used" in q033). This choice was made after seeing test results, so its score is somewhat optimistic. This is the baseline-first lesson: at 28 tables, retrieval only pays off once memory and follow-ups matter.

| Choice | Why | Alternative considered |
|---|---|---|
| LangGraph | graph loop with explicit limits; checkpointer gives chat memory for free | Pydantic AI, smolagents |
| FastMCP server | one safe plug for all data tools; any MCP client can use it | plain Python functions |
| Qdrant (local) + FastEmbed | dense + BM25 + RRF built in; no server; free local models | pgvector, Chroma |
| Neo4j | shortest join paths and look-alikes are one Cypher query | a hand-coded join list |
| DuckDB | fast local warehouse; read-only mode for safety | SQLite, Postgres |
| gpt-5-mini / gpt-5 | cheap model for every version; a larger model as the blind grader | one model for both |

**Limitations:** AI-drafted, template-based benchmark (frozen and cross-checked, but not written blind); simulated "now"; estimated value is not revenue; bigger-catalog runs are 1 run each; human/judge kappa (0.65) rests on only 2 "no" grades among 30; Langfuse not used; unit tests and CI added after the final runs. **Next:** the "Why" stage (metric trees in the map, multi-agent supervisor).